

Unit 4 - Python SQLite

Python SQLite3 module is used to integrate the SQLite database with Python. It provides a straightforward and simple-to-use interface for interacting with SQLite databases.

Installation

SQLite3 can be integrated with Python using `sqlite3` module and it provides an SQL interface. You do not need to install this module separately because it is shipped by default along with Python version 2.5.x onwards.

To use `sqlite3` module, you must first create a connection object that represents the database and then create a cursor object, which will help you in executing all the SQL statements.

SQLite Methods

Following are important `sqlite3` module methods, which can suffice your requirement to work with SQLite database from your Python program.

1. `connect()`

To use SQLite3 in Python, first we have to import the `sqlite3` module and then create a connection object. Connection object allows to connect to the database and will let us execute the SQL statements.

Creating Connection object using the `connect()` function:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
```

This will create a new file with the name 'mydatabase.db'.

2. `cursor()`

To execute SQLite statements in Python, we need a cursor object. We can create it using the `cursor()` method.

The SQLite3 cursor is a method of the connection object. To execute the SQLite3 statements, you should establish a connection at first and then create an object of the cursor using the connection object as follows:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
```

Now we can use the cursor object to call the `execute()` method to execute any SQL queries.

3. `execute()`

Once the database and connection object is created, we can create a table using CREATE TABLE statement. Then we execute the CREATE TABLE statement by calling `cur.execute(...)`.

```
Eg:
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute(" CREATE TABLE movie(title text, year int, score real) ")
```

Here, 'movie' is the table name with columns title, year and score. Since SQLite is flexible, we can just use column names in the table declaration, specifying the data types is optional.

4. Close()

Once we are done with our database, it is a good practice to close the connection. We can close the connection by using the *close()* method.

To close a connection, use the connection object and call the *close()* method as follows:

```
con = sqlite3.connect('mydatabase.db')
#program statements
con.close()
```

SQLite database operations

I. Creating and connecting to Database

When you create a connection with SQLite, that will create a database file automatically if it doesn't already exist. This database file is created on disk with the connect function.

Following Python code shows how to connect to an existing database. If the database does not exist, then it will be created and finally a database object will be returned.

```
import sqlite3
conn = sqlite3.connect('test.db')
print ("Opened database successfully")
```

Output:
Opened database successfully

II. Create Table

To create a table in SQLite3, you can use the Create Table query in the *execute()* method. Consider the following steps:

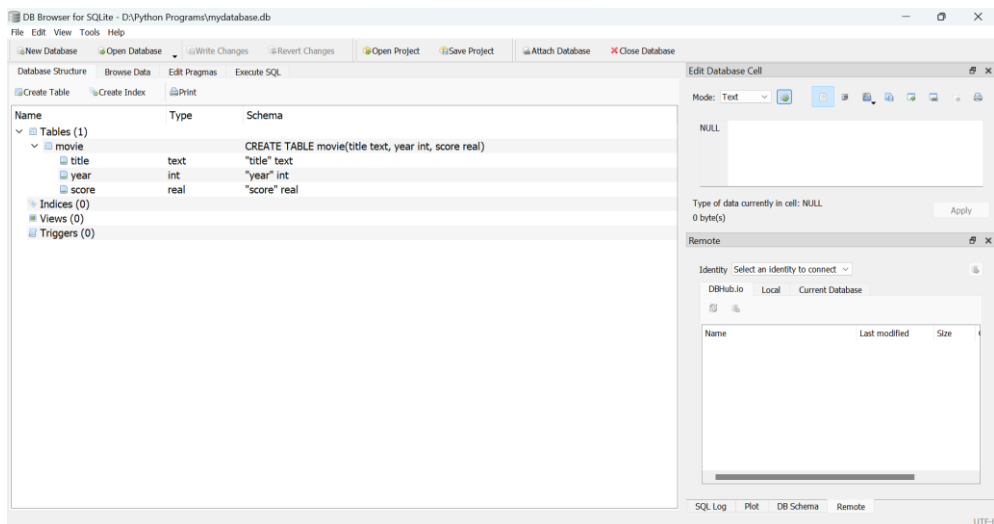
1. Create a connection object.
2. From the connection object, create a cursor object.
3. Using the cursor object, call the execute method with create table query as the parameter.

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("CREATE TABLE movie(title text, year int, score real)")
con.commit()
con.close()
```

In the above code, it establishes a connection and creates a cursor object to execute the create table statement.

The *commit()* method saves all the changes we make.

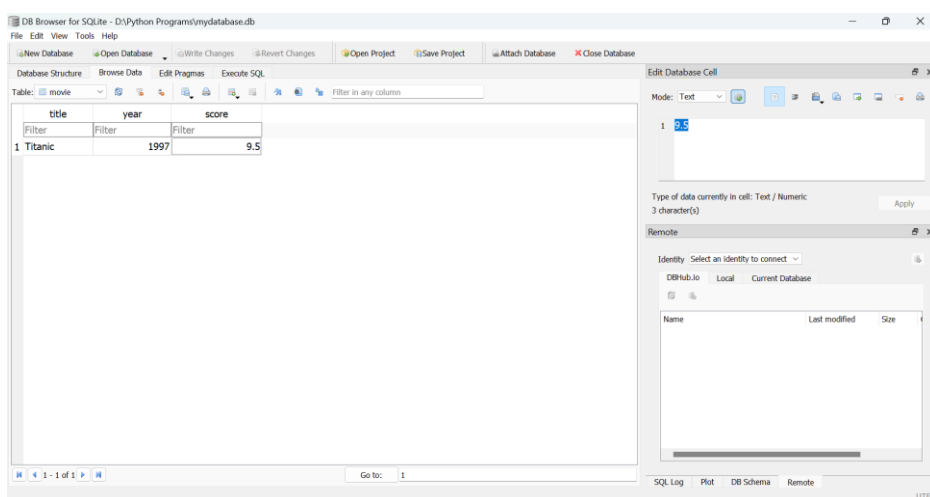
To check if our table is created, you can use the [DB browser for SQLite](https://sqlitebrowser.org/dl/) (<https://sqlitebrowser.org/dl/>) to view your table. Open your mydatabase.db file with the program, and you should see your table:



III. Insert in Table

To insert data in a table, we use the INSERT INTO statement. Consider the following line of code:

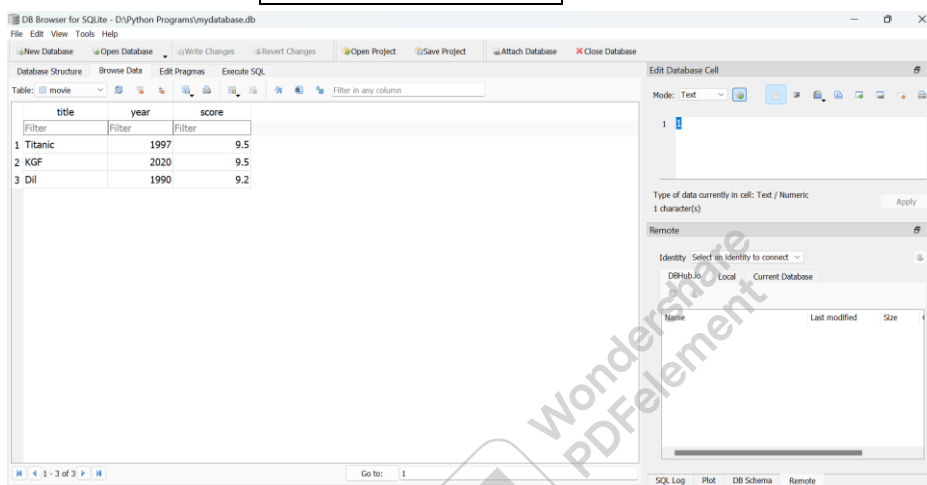
```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("CREATE TABLE movie(title text, year int, score real)")
cursorObj.execute("INSERT INTO movie VALUES ('Titanic',1997, 9.5)")
con.commit()
con.close()
```



We can also pass values to an INSERT statement in the `execute()` method. You can use the question mark (?) as a placeholder for each value. The syntax of the INSERT will be like the following:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
m=input("Movie Name :")
y=int(input("Year :"))
s=float(input("Score :"))
cursorObj.execute("INSERT INTO MOVIE VALUES (?,?,:) ",(m,y,s))
con.commit()
con.close()
```

User Input:
Movie Name :Dil
Year :1990
Score :9.2



IV. Update Table

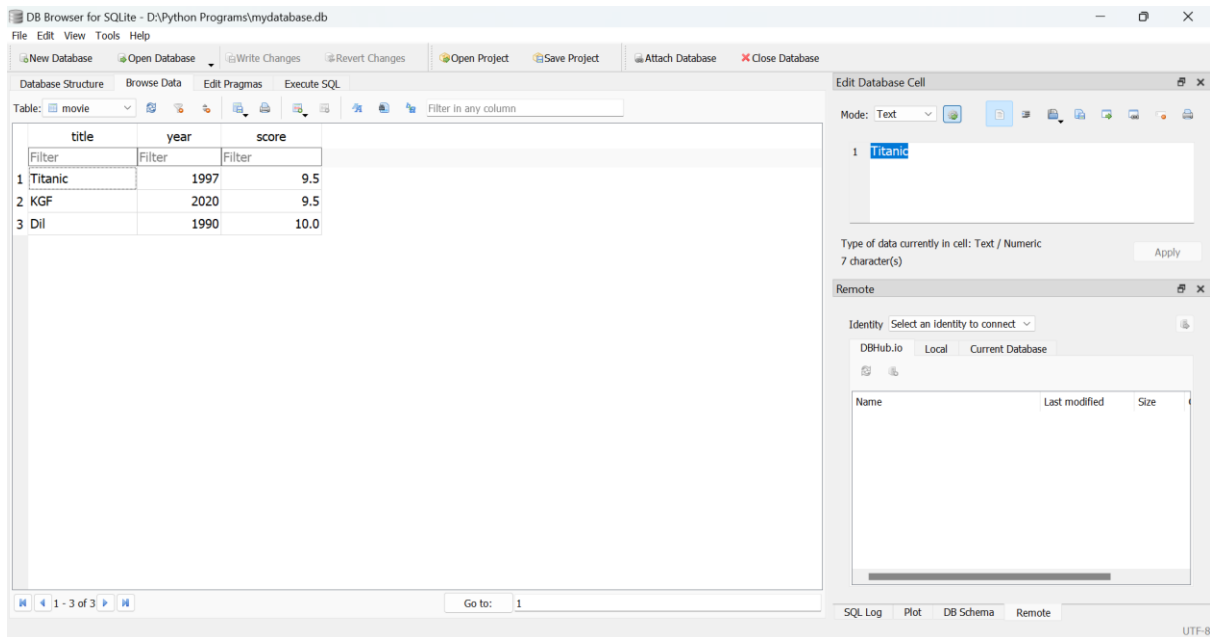
To update the table, simply create a connection, then create a cursor object using the connection and finally use the UPDATE statement in the *execute()* method.

Suppose that we want to update the score with the movie title Dil. For updating, we will use the UPDATE statement and for the movie whose title equals Dil. We will use the WHERE clause as a condition to select this employee.

Consider the following code:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("UPDATE MOVIE SET SCORE=10 WHERE TITLE='Dil' ")
con.commit()
con.close()
```

This will change the score for the movie Dil:



V. Select statement

You can use the select statement to select data from a particular table. If you want to select all the columns of the data from a table, you can use the asterisk (*). The syntax for this will be as follows:

```
select * from table_name
```

In SQLite3, the SELECT statement is executed in the execute method of the cursor object. For example, select all the columns of the movie table, run the following code:

```
cursorObj.execute("SELECT * FROM movie ")
```

If you want to select a few columns from a table, then specify the columns like the following:

```
select column1, column2 from tables_name
```

For example,

```
cursorObj.execute( "SELECT title, year FROM movie")
```

The select statement selects the required data from the database table, and if you want to fetch the selected data, the *fetchall()* method of the cursor object is used. We will demonstrate this in the next section.

Fetch all data

To fetch the data from a database, we will execute the SELECT statement and then will use the *fetchall()* method of the cursor object to store the values into a variable. After that, we will loop through the variable and print all values.

The code will be like this:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("SELECT * FROM movie ")
records=cursorObj.fetchall()
for row in records:
    print(row)
con.commit()
con.close()
```

The above code will print out the records in our database as follows:

```
Output:
('Titanic', 1997, 9.5)
('KGF', 2020, 9.5)
('Dil', 1990, 10.0)
```

You can also use the *fetchall()* in one line as follows:

```
[print(row) for row in cursorObj.fetchall()]
```

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]
con.commit()
con.close()
```

If you want to fetch specific data from the database, you can use the WHERE clause.

VI. Delete

In SQLite database we use the following syntax to delete data from a table:

```
DELETE FROM table_name [WHERE Clause]
```

- Import the required module.
- Create a connection object with the database using `connect()`.
- Create a Cursor object by calling the `cursor()`.
- Finally, use `execute()` method by passing a DELETE statement as a parameter to it.

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
#Deleting
cursorObj.execute("Delete from movie where title='Titanic' ")
con.commit()
con.close()
```

VII. Drop table

You can drop/delete a table using the DROP statement. The syntax of the DROP statement is as follows:

```
drop table table_name
```

To drop a table, the table should exist in the database. Therefore, it is recommended to use “*if exists*” with the drop statement as follows:

```
drop table if exists table_name
```

For example,

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute(" DROP TABLE IF EXISTS MOVIE ")
con.commit()
con.close()
```

#Program to demonstrate a database operations create, insert, select, update, delete and drop

```
#importing the module
import sqlite3

#create connection object
con = sqlite3.connect('mydatabase.db')

#crete a cursor
cursorObj = con.cursor()

#creating the table
cursorObj.execute("CREATE TABLE movie(title text, year int, score real)")

#inserting the data
cursorObj.execute("INSERT INTO movie VALUES ('Titanic',1997, 9.5)")
cursorObj.execute("INSERT INTO movie VALUES ('KGF',2020, 9.1)")
cursorObj.execute("INSERT INTO movie VALUES ('Dil',1990, 8.5)")

#Print the Initial Data
print("Initial Data...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]

#Updating
cursorObj.execute("UPDATE MOVIE SET SCORE=10 WHERE TITLE='Dil' ")

#Print the after updating
print("After updating...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]
```

```
#Deleting
cursorObj.execute("Delete from movie where title='Titanic' ")

#Print the after Deleting
print("After deleting...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]

#Drop the table
cursorObj.execute(" DROP TABLE IF EXISTS MOVIE ")

#commit changes in the database
con.commit()

#close the connection
con.close()
```

Output
Initial Data...
('Titanic', 1997, 9.5)
('KGF', 2020, 9.1)
('Dil', 1990, 8.5)

After updating...
('Titanic', 1997, 9.5)
('KGF', 2020, 9.1)
('Dil', 1990, 10.0)

After deleting...
('KGF', 2020, 9.1)
('Dil', 1990, 10.0)

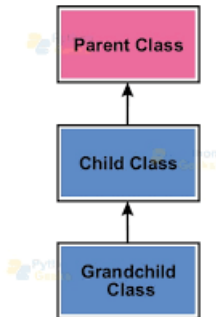


Unit 3 - Object Oriented Programming

Multilevel Inheritance

In Python, we can also derive a class from the derived class. This form of inheritance is known as multilevel inheritance.

Multilevel Inheritance



Here's the **syntax of the multilevel inheritance**,

```
class SuperClass:
    # Super class code here

class DerivedClass1(SuperClass):
    # Derived class 1 code here

class DerivedClass2(DerivedClass1):
    # Derived class 2 code here
```

Here, the DerivedClass1 class is derived from the SuperClass class, and the DerivedClass2 class is derived from the DerivedClass1 class.

#Program to demonstrate multilevel inheritance

```
class Manager:
    def final_review(self):
        print("Final Review")

class Reviewer(Manager):
    def review(self):
        print("Reviewing...")

class Writer(Reviewer):
    def writes(self):
        print("Writes the code")

obj = Writer()
obj.final_review()
obj.review()
obj.writes()
```

Output:

```
Final Review
Reviewing...
Writes the code
```

#Program to demonstrate super() function in multilevel inheritance

```
class Parent:
    def __init__(self):
        print('Parent - Hii')
    def age(self, a):
        print('Printing the age (Parent): ', a)

class Child(Parent):
    def __init__(self):
        print('Child - Hii')
```

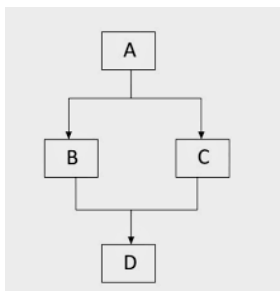
```
        super().__init__()\n    def age(self, a):\n        print('Printing the age(Child): ', a)\n        super().age(a + 30)\n\nclass GrandChild(Child):\n    def __init__(self):\n        print('Grand Child - Hii')\n        super().__init__()\n    def age(self, a):\n        print('Printing the age(Grand Child): ', a)\n        super().age(a + 25)\n\n# Main function\nif __name__ == '__main__':\n    obj = GrandChild()\n    obj.age(10)
```

Output

```
Grand Child - Hii\nChild - Hii\nParent - Hii\nPrinting the age(Grand Child): 10\nPrinting the age(Child): 35\nPrinting the age (Parent): 65
```

Multipath Inheritance

When a class is derived from two or more classes which are derived from the same base class then such type of inheritance is called multipath inheritance.



Here class 'D' derived from classes 'B' and 'C', which are derived from same base class 'A'.

Here's the **syntax of the multipath inheritance**,

```
class ClassA:\n    # Super class code here\n\nclass ClassB(ClassA):\n    # Derived class B code here\n\nclass ClassC(ClassA):\n    # Derived class C code here\n\nclass ClassD(ClassB, ClassC):\n    # Derived class D code here
```

#Program to demonstrate Multipath Inheritance

```
class University:
    def __init__(self):
        print("Constructor of the Base class")
    def display(self):
        print(f"The University Class display method")

class Course(University):
    def __init__(self):
        print("Constructor of the Child Class 1 of Class University")
        super().__init__()
    def display(self):
        print(f"The Course Class display method")
        super().display()

class Branch(University):
    def __init__(self):
        print("Constructor of the Child Class 2 of Class University")
        super().__init__()
    def display(self):
        print(f"The Branch Class display method ")
        super().display()

class Student(Course, Branch):
    def __init__(self):
        print("Constructor of Child class of Course and Branch is called")
        super().__init__()
    def display(self):
        print(f"The Student Class display method")
        super().display()

# Object Instantiation:
ob = Student() # Object named ob of the class Student.
print()
ob.display() # Calling the display method of Student class.
```

Output:

Constructor of Child class of Course and Branch is called
Constructor of the Child Class 1 of Class University
Constructor of the Child Class 2 of Class University
Constructor of the Base class

The Student Class display method
The Course Class display method
The Branch Class display method
The University Class display method

12.2.4 JSON versus XML

Since both JSON and XML are widely used as data interchange formats, we will try to draw a comparison between them.

- XML is more expressive than JSON. However, XML suffers from the frequent use of tags, whereas JSON is much more compact.
- XML is more complex than JSON.
- Both JSON and XML can be used with most of the programming languages. But the way the programming languages handle these two format is different. When you are working with XML, its data format does not directly translate to a programming language data structure, thus forcing to work with two systems whose data structures are different. The objects and arrays used in JSON are inherently compatible with most of the programming languages' data structures, which eases the use of JSON format in a programming language.
- XML has XSLT (Extensible Stylesheet Language Transformations) specification, which may be used to apply a style to an XML document. JSON does not have any such thing.

12.3 NumPy with Python

NumPy is the fundamental package for scientific computing with Python. It stands for "Numerical Python." It supports:

- N-dimensional array object
- Broadcasting functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as a multi-dimensional container to store generic data. Arbitrary data types can also be defined. This allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

NumPy's main object is the homogeneous multidimensional array. An array is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers and represented by a single variable. NumPy's array class is called *ndarray*. It is also known by the alias *array*.

In NumPy arrays, the individual data items are called *elements*. All elements of an array should be of the same type. Arrays can be made up of any number of dimensions. In NumPy, dimensions are called axes. Each dimension of an array has a length which is the total number of elements in that direction. The size of an array is the total number of elements contained in an array in all the dimension. The size of NumPy arrays are fixed; once created it cannot be changed again.

For example, [FIGURE 12.3](#) shows the axes (or dimensions) and lengths of two example arrays; 12.3(a) is a one-dimensional array and 12.3(b) is a two-dimensional array. A one-dimensional array has one axis indicated by Axis-0. That axis has five elements in it, so we say it has a length of five. A two-dimensional array is made up of rows and columns.

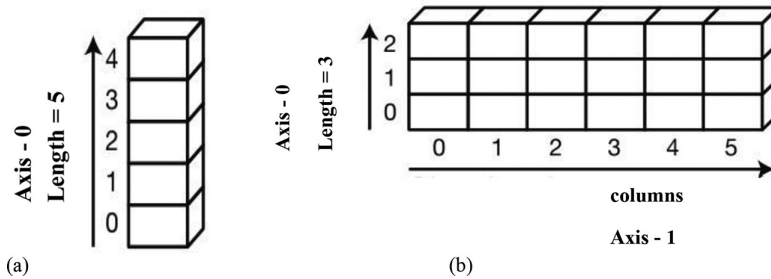


FIGURE 12.3
Dimensions of NumPy Array.

All the rows are indicated by Axis-0 and all the columns are indicated by Axis-1. In a two-dimensional array, Axis-0 has three elements in it, so its length is three and Axis-1 has six elements in it, so its length is six. Notice that for each axis, the indexes range from 0 to length - 1. Array indexes are 0-based. That is, if the length of a dimension is n , the index values range from 0 to $n - 1$.

In order to use NumPy in your program, you need to import NumPy. For example,

```
import numpy as np
```

numpy is usually renamed as np.

12.3.1 NumPy Arrays Creation Using `array()` Function

You can create a NumPy array from a regular Python list or tuple using the `np.array()` function. The type of the resulting array is deduced from the type of the elements. For example,

1. `>>> import numpy as np`
2. `>>> int_number_array = np.array([1,2,3,4])`
3. `>>> int_number_array`
`array([1, 2, 3, 4])`
4. `>>> type(int_number_array)`
`<class 'numpy.ndarray'>`
5. `>>> int_number_array.dtype`
`dtype('int32')`
6. `>>> float_number_array = np.array([9.1, 8.1, 8.8, 3.0])`
7. `>>> float_number_array.dtype`
`dtype('float64')`
8. `>>> two_dimensional_array_list = np.array([[1,2,3], [4,5,6]])`
9. `>>> two_dimensional_array_list`
`array([[1, 2, 3],`
`[4, 5, 6]])`
10. `>>> two_dimensional_array_tuple = np.array(((1,2,3), (4,5,6)))`

```

11. >>> two_dimensional_array_tuple
    array([[1, 2, 3],
           [4, 5, 6]])
12. >>> array_dtype = np.array([1,2,3,4], dtype = np.float64)
13. >>> array_dtype
    array([1., 2., 3., 4.])
14. >>> array_dtype.dtype
    dtype('float64')

```

Import the *numpy* library into your program ①. Pass a list of items to the *np.array()* function and assign the result to *int_number_array* object ②. In the output, you can see all the elements are placed within an iterable object of *array* class and is a one-dimensional array. The *int_number_array* object belongs to *numpy.ndarray* class ④. NumPy provides a large set of numeric datatypes that you can use to construct arrays. NumPy tries to guess a datatype when you create an array, but functions that construct arrays also usually include an optional argument to explicitly specify the datatype. The type of *int_number_array* is *dtype('int32')* ⑤. The type of *float_number_array* ⑥ is *dtype('float64')* ⑦. The *np.array()* function takes either single list or tuple as an argument. If you want to specify multiple lists or tuples, then pass it as nested lists ⑧–⑨ or nested tuples ⑩–⑪. Here, *two_dimensional_array_list* and *two_dimensional_array_tuple* are examples for two-dimensional arrays. You can also explicitly specify the data type of array by assigning type values like *np.float64*, *np.int32*, and others to a *dtype* attribute and pass it as a second argument to *np.array()* function ⑫–⑭.

12.3.2 Array Attributes

The contents of *ndarray* can be accessed and modified by indexing or slicing the array and via the methods and attributes of the *ndarray*. The more important attributes of *ndarray* object are (TABLE 12.2).

TABLE 12.2

Array Attributes

ndarray Attributes	Description
<i>ndarray.ndim</i>	Gives the number of axes or dimensions in the array
<i>ndarray.shape</i>	Gives the dimensions of the array. For an array with n rows and m columns, shape will be a tuple of integers (n, m).
<i>ndarray.size</i>	Gives the total number of elements of the array.
<i>ndarray.dtype</i>	Gives an object describing the type of the elements in the array. One can create or specify <i>dtype</i> 's using standard Python types. Additionally, NumPy provides its own types like <i>np.int32</i> , <i>np.int16</i> , <i>np.float64</i> , and others.
<i>ndarray.itemsize</i>	Gives the size of each element of the array in bytes.
<i>ndarray.data</i>	Gives the buffer containing the actual elements of the array. Normally, we will not use this attribute because we will access the elements in an array using indexing facilities.

Note: In your code, replace *ndarray* with you Python NumPy *ndarray* object name.

For example,

```
1. >>> import numpy as np
2. >>> array_attributes = np.array([[10, 20, 30], [14, 12, 16]])
3. >>> array_attributes.ndim
2
4. >>> array_attributes.shape
(2, 3)
5. >> array_attributes.size
6
6. >>> array_attributes.dtype
dtype('int32')
7. >>> array_attributes.itemsize
4
8. >>> array_attributes.data
<memory at 0x000001E61DB963A8>
```

Various ndarray attributes ①–⑧.

12.3.3 NumPy Arrays Creation with Initial Placeholder Content

Often, the elements of an array are initially unknown, but its size is known. Hence, NumPy offers several functions to create arrays with initial placeholder content (TABLE 12.3). These minimize the necessity of growing arrays, an expensive operation.

TABLE 12.3

NumPy Arrays Creation Functions

Function Name	Description
<code>np.zeros()</code>	Creates an array of zeros
<code>np.ones()</code>	Creates an array of ones
<code>np.empty()</code>	Creates an empty array
<code>np.full()</code>	Creates a full array
<code>np.eye()</code>	Creates an identity matrix
<code>np.random.random()</code>	Creates an array with random values
<code>np.arange()</code>	The syntax for <code>arange()</code> is, <code>np.arange([start,]stop, [step,][dtype=None])</code> Returns evenly spaced values within a given interval where <i>start</i> (a number and optional) is the start of interval and its default value is zero, <i>stop</i> (a number) is the end of interval, and <i>step</i> (a number and is optional) is the spacing between the values and <i>dtype</i> is the type of output array.

(Continued)

TABLE 12.3 (Continued)

NumPy Arrays Creation Functions

Function Name	Description
<code>np.linspace()</code>	The syntax for <code>linspace</code> is, <pre><i>numpy.linspace(start, stop, num=50, dtype=None)</i></pre> Returns evenly spaced numbers over a specified interval where <i>start</i> is the starting value of the sequence, <i>stop</i> is the end value of the sequence, and <i>num</i> (an integer and optional) is the number of samples to generate. Default is 50. Must be non-negative. The optional <i>dtype</i> is the type of the output array.

For example,

1.

```
>>> import numpy as np
```
2.

```
>>> np.zeros((2,3))
```



```
array([[0., 0., 0.],
```



```
       [0., 0., 0.]])
```
3.

```
>>> np.ones((3,4))
```



```
array([[1., 1., 1., 1.],
```



```
       [1., 1., 1., 1.],
```



```
       [1., 1., 1., 1.]])
```
4.

```
>>> np.empty((2,3))
```



```
array([[0., 0., 0.],
```



```
       [0., 0., 0.]])
```
5.

```
>>> np.full((3,3),2)
```



```
array([[2, 2, 2],
```



```
       [2, 2, 2],
```



```
       [2, 2, 2]])
```
6.

```
>>> np.eye(2,2)
```



```
array([[1., 0.],
```



```
       [0., 1.]])
```
7.

```
>>> np.random.random((2,2))
```



```
array([[0.95022839, 0.23253555],
```



```
       [0.843828 , 0.57976282]])
```
8.

```
>>> np.arange(10, 30, 5)
```



```
array([10, 15, 20, 25])
```
9.

```
>>> np.arange(0, 2, 0.3)
```



```
array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])
```
10.

```
>>> np.linspace(0, 2, 9)
```



```
array([0. , 0.25, 0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2. ])
```


Various NumPy functions to create arrays ①–⑩. When the *arange()* function is used with floating point arguments, it is generally not possible to predict the number of elements obtained, due to the finite floating point precision ⑨. For this reason, it is usually better to use the *linspace()* function to which you can pass an argument specifying the number of elements you want to generate instead of the *step*. In ⑩, the *linspace()* function produces nine data elements between zero and two.

12.3.4 Integer Indexing, Array Indexing, Boolean Array Indexing, Slicing and Iterating in Arrays

One-dimensional arrays can be indexed, sliced, and iterated over, much like lists and other Python sequences.

```
1. >>> import numpy as np
2. >>> a = np.arange(5)
3. >>> a
array([0, 1, 2, 3, 4])
4. >>> a[2]
2
5. >>> a[2:4]
array([2, 3])
6. >>> a[4:2] = -999
7. >>> a
array([-999, 1, -999, 3, 4])
8. >>> a[::-1]
array([ 4, 3, -999, 1, -999])
9. >>> for each_element in a:
10. ...     print(each_element)
-999
1
-999
3
4
```

Indexing, slicing, and iterating operations on one-dimensional NumPy arrays ①–⑩.

For multi-dimensional arrays you can specify an index or slice per axis. For example,

```
1. >>> import numpy as np
2. >>> a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
```

		COLUMNS				
		0	1	2	3	
ROWS	0	1	2	3	4	 a[0, 0]
	1	5	6	7	8	 a[1, 3]
	2	9	10	11	12	 a[2, 1]

3.

```
>>> a[1, 3]
```


8
4.

```
>>> a[:2, 1:3]
```


array([[2, 3],
 [6, 7]])
5.

```
>>> lower_axes = a[1, :]
```
6.

```
>>> lower_axes
```


array([5, 6, 7, 8])
7.

```
>>> lower_axes.ndim
```


1
8.

```
>>> same_axes = a[1:2, :]
```
9.

```
>>> same_axes
```


array([[5, 6, 7, 8]])
10.

```
>>> same_axes.ndim
```


2
11.

```
>>> a[:, 1]
```


array([2, 6, 10])
12.

```
>>> a[:, 1:2]
```


array([[2],
 [6],
 [10]])
13.

```
>>> for row in a:
```
14.

```
...     print(row)
```


[1 2 3 4]
[5 6 7 8]
[9 10 11 12]
15.

```
>>> for each_element in a.flat:
```

```

16. ...     print(each_element)
17.
18. 1
19. 2
20. 3
21. 4
22. 5
23. 6
24. 7
25. 8
26. 9
27. 10
28. 11
29. 12

```

Use integer indexing to pull out the data elements present in row 1 and column 3 ③. You can also mix integer indexing with slice indexing. In ④, pull out the subarray consisting of row 0 and row 1 and column 1 and column 2 having a shape of (2, 2). Display the elements present in all the columns of row 1 ⑤–⑥, ⑧–⑨. Mixing integer indexing with slices yields an array of lower axes ⑦. Using slices in all the axes of an array yields an array of the same axes as the original array ⑩. Display all the elements present in all the rows of column 1 ⑪–⑫. Iterating over multi-dimensional arrays is done with respect to the first axis ⑬–⑭. However, if one wants to perform an operation on each element in the array, one can use the *flat* attribute, which is an iterator over all the elements of the array ⑮–⑯.

```

1. >>> import numpy as np
2. >>> a = np.array([[1, 2], [3, 4], [5, 6]])
3. >>> a
   array([[1, 2],
          [3, 4],
          [5, 6]])
4. >>> a[[0, 1, 2], [0, 1, 0]]
   array([1, 4, 5])

```

Integer array indexing allows you to construct arbitrary arrays using the data from another array ②. Here in code line ④, it is used to construct an array from another array ③.

```

a[[0, 1, 2], [0, 1, 0]]

```

Elements found in (0, 0), (1, 1), and (2, 0) index are pulled out and displayed ④.

```

1. >>> import numpy as np
2. >>> a = np.array([[11, 12], [13, 14], [15, 16]])

```

```
3. >>> a
   array([[11, 12],
          [13, 14],
          [15, 16]])
4. >>> a[a > 13]
   array([14, 15, 16])
```

Boolean array indexing lets you select the elements of an array that satisfy some condition. With Boolean array indexing, you explicitly choose which items in the array you want and which ones you do not want. With Boolean array indexing of $a[a > 13]$ ④, elements greater than 13 in the array a are displayed as a one-dimensional array.

12.3.5 Basic Arithmetic Operations on NumPy Arrays

Basic mathematical functions perform element-wise operation on arrays and are available both as operator overloads and as functions in the NumPy module. For example,

```
1. >>> import numpy as np
2. >>> a = np.array( [20, 30, 40, 50] )
3. >>> b = np.arange(4)
4. >>> b
   array([0, 1, 2, 3])
5. >>> a + b
   array([20, 31, 42, 53])
6. >>> np.add(a, b)
   array([20, 31, 42, 53])
7. >>> a - b
   array([20, 29, 38, 47])
8. >>> np.subtract(a, b)
   array([20, 29, 38, 47])
9. >>> A = np.array( [[1, 1], [6, 1]] )
10. >>> B = np.array( [[2, 8], [3, 4]] )
11. >>> A * B
   array([[2, 8],
          [18, 4]])
12. >>> np.multiply(A, B)
   array([[ 2, 8],
          [18, 4]])
13. >>> A / B
   array([[0.5 , 0.125],
          [2. , 0.25 ]])
```

In the above code, the shape of *array_1* ② is (2, 3) ④ and *array_2* ③ is (5,) ⑤.

```
array_1.shape → (2, 3)
```

```
array_2.shape → (5,)
```

Since *array_2* has less dimension compared to *array_1*, according to Rule 1, *array_2* is padded with 1's on its left. Now the shape of *array_2* becomes (1, 5).

```
array_1.shape → (2, 3)
```

```
array_2.shape → (1, 5)
```

Next, according to Rule 2, the shape of *array_2* having "1" in the first dimension is stretched to match the highest shape along that dimension of *array_1*. Thus, the shape of *array_2* becomes (2, 5).

```
array_1.shape → (2, 3)
```

```
array_2.shape → (2, 5)
```

But the shapes of both the arrays differ and according to Rule 3 the addition operation fails in this case ⑥.

12.4 Pandas

pandas is a Python library that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

The two primary data structures of *pandas*, *Series* (one-dimensional) and *DataFrame* (two-dimensional), handle the vast majority of typical-use cases in finance, statistics, social science, and many areas of engineering. *pandas* is built on top of NumPy and is intended to integrate well within a scientific computing environment with many other third-party libraries.

pandas is well suited for inserting and deleting columns from *DataFrame*, for easy handling of missing data (represented as NaN), explicitly aligning data to a set of labels, converting data in other Python and NumPy data structures into *DataFrame* objects, intelligent label-based slicing, indexing, and subsetting of large data sets, merging and joining of data sets, and flexible reshaping. Additionally, it has robust input/output tools for loading data from CSV files, Excel files, databases, and other formats. You have to import a *pandas* library to make use of various functions and data structures defined in *pandas*.

```
import pandas as pd
```

pandas is usually renamed as *pd*.

12.4.1 Pandas Series

Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). The axis labels are collectively referred to as the *index*. Pandas Series is created using *series()* method and its syntax is,

```
s = pd.Series(data, index=None)
```

Here, *s* is the Pandas Series, *data* can be a Python *dict*, a *ndarray*, or a scalar value (like 5). The passed index is a list of axis labels. Both integer and label-based indexing are supported. If the index is not provided, then the index will default to *range(n)* where *n* is the length of *data*. For example,

Create Series from ndarrays

```
1. >>> import numpy as np
2. >>> import pandas as pd
3. >>> s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])
4. >>> type(s)
    <class 'pandas.core.series.Series'>
5. >>> s
    a  -0.367740
    b   0.855453
    c  -0.518004
    d  -0.060861
    e  -0.277982
    dtype: float64
6. >>> s.index
    Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
7. >>> s.values
    array([-0.367740,  0.855453, -0.518004, -0.060861, -0.277982])
8. >>> pd.Series(np.random.randn(5))
    0    0.334947
    1   -2.184006
    2   -0.209440
    3   -0.492398
    4   -1.507088
    dtype: float64
```

Import NumPy and pandas libraries ①–②. Create a series using *ndarray* which is NumPy's array class using *Series()* method ③ which returns a Pandas Series type *s* ④. You can also specify axis labels for *index*, i.e., *index=['a', 'b', 'c', 'd', 'e']* ⑤. When data is a *ndarray*, the *index* must be the same length as data. In series *s* ⑤, by default the type of values of all the elements is *dtype: float64*. You can find out the index for a series using *index* attribute ⑥.

The *values* attribute returns a *ndarray* ⑦ containing only values, while the axis labels are removed. If no labels for the index is passed, one will be created having a range of index values [0,..., len(data) - 1] ⑧.

Create Series from Dictionaries

```
1. >>> import numpy as np
2. >>> import pandas as pd
3. >>> d = {'a' : 0., 'b' : 1., 'c' : 2.}
4. >>> pd.Series(d)
   a    0.0
   b    1.0
   c    2.0
  dtype: float64
5. >>> pd.Series(d, index=['b', 'c', 'd', 'a'])
   b    1.0
   c    2.0
   d   NaN
   a    0.0
  dtype: float64
```

Series can be created from the dictionary. Create a dictionary ③ and pass it to *Series()* method ④. When a series is created using dictionaries, by default the keys will be index labels. While creating series using a dictionary, if labels are passed for the *index*, the values corresponding to the labels in the index will be pulled out ⑤. The order of index labels will be preserved. If a value is not associated for a label, then *NaN* is printed. *NaN* (not a number) is the standard missing data marker used in pandas.

Create Series from Scalar data

```
1. >>> import numpy as np
2. >>> import pandas as pd
3. >>> pd.Series(5., index=['a', 'b', 'c', 'd', 'e'])
   a    5.0
   b    5.0
   c    5.0
   d    5.0
   e    5.0
  dtype: float64
```

You can create a Pandas Series from scalar value. Here scalar value is five ③. If data is a scalar value, an index must be provided. The value will be repeated to match the length of the index.

Series Indexing and Slicing

```
1. >>> import numpy as np
2. >>> import pandas as pd
```

```
3. >>> s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])
4. >>> s
a    0.481557
b    2.053330
c   -1.799993
d   -0.396880
e   -1.270751
dtype: float64
5. >>> s[0]
0.48155677569897515
6. >>> s[1:3]
b    2.053330
c   -1.799993
dtype: float64
7. >>> s[:3]
a    0.481557
b    2.053330
c   -1.799993
dtype: float64
8. >>> s[s > .5]
b    2.05333
dtype: float64
9. >>> s[[4, 3, 1]]
e   -1.270751
d   -0.396880
b    2.053330
dtype: float64
10. >>> s['a']
0.48155677569897515
11. >>> s['e']
-1.270750548062543
12. >>> 'e' in s
True
13. >>> 'f' in s
False
```

You can provide index ⑤ or slice data by index numbers ⑥–⑦ in a Pandas Series ③–④. You can also specify a Boolean array indexing for Pandas Series ⑧. Multiple indices are specified as a list in ⑨. The index can be an integer value or a label ⑩. Values associated with labeled index are extracted and displayed ⑪–⑫. Check for the presence of a label in Series using *in* operator ⑬–⑭.

Working with Text Data

The Pandas Series supports a set of string processing methods that make it easy to operate on each element of the array. These methods are accessible via the *str* attribute and they generally have the same name as that of the built-in Python string methods.

```
1. >>> import numpy as np
2. >>> import pandas as pd
3. >>> empires_ds = pd.Series(["Vijayanagara", "Roman", "Chola", "Mongol",
    "Akkadian"])
4. >>> empires_ds.str.lower()
0 vijayanagara
1 roman
2 chola
3 mongol
4 akkadian
dtype: object
5. >>> empires_ds.str.upper()
0 VIJAYANAGARA
1 ROMAN
2 CHOLA
3 MONGOL
4 AKKADIAN
dtype: object
6. >>> empires_ds.str.len()
0 11
1 5
2 5
3 6
4 8
dtype: int64
7. >>> tennis_ds = pd.Series(['Seles ', ' Graph ', ' Williams '])
8. >>> tennis_ds.str.strip()
0 Seles
1 Graph
2 Williams
dtype: object
9. >>> tennis_ds.str.contains(' ')
0 True
1 True
```

```
2 True
dtype: bool
10. >>> marvel_ds = pd.Series(['Thor_loki', 'Thor_Hulk', 'Gamora_Storm'])
11. >>> marvel_ds.str.split('_')
0 [Thor, loki]
1 [Thor, Hulk]
2 [Gamora, Storm]
dtype: object
12. >>> planets = pd.Series(["Venus", "Earth", "Saturn"])
13. >>> planets.str.replace("Earth", "Mars")
0 Venus
1 Mars
2 Saturn
dtype: object
14. >>> letters_ds = pd.Series(['a', 'b', 'c', 'd'])
15. >> letters_ds.str.cat(sep=',')
'a,b,c,d'
16. >>> names_ds = pd.Series(['Jahnavi', 'Adelmo', 'Pietro', 'Alejandro'])
17. >>> names_ds.str.count('e')
0 0
1 1
2 1
3 1
dtype: int64
18. >>> names_ds.str.startswith('A')
0 False
1 True
2 False
3 True
dtype: bool
19. >>> names_ds.str.endswith('O')
0 False
1 False
2 False
3 False
dtype: bool
20. >>> names_ds.str.find('J')
0 0
1 -1
```

```
2 -1
3 -1
dtype: int64
```

Various string methods to operate with Pandas Series is discussed ①–②⑩.

12.4.2 Pandas DataFrame

DataFrame is a two-dimensional, labeled data structure with columns of potentially different types. You can think of it like a spreadsheet or database table, or a *dict* of Series objects. It is generally the most commonly used pandas object. DataFrame accepts many different kinds of input like *Dict* of one-dimensional *ndarrays*, lists, *dicts*, or Series, two-dimensional *ndarrays*, structured or record *ndarray*, a dictionary of Series, or another DataFrame.

```
df = pd.DataFrame(data=None, index=None, columns=None)
```

Here, *df* is the DataFrame and *data* can be NumPy *ndarray*, *dict*, or DataFrame. Along with the *data*, you can optionally pass an *index* (row labels) and *columns* (column labels) attributes as arguments. If you pass an *index* and/or *columns*, you are guaranteeing the *index* and/or *columns* of the resulting DataFrame. Both *index* and *columns* will default to *range(n)* where *n* is the length of *data*, if they are not provided. When the *data* is a dictionary and *columns* are not specified, then the DataFrame column labels will be dictionary's *keys*.

Create DataFrame from Dictionary of Series/Dictionaries

```
1. >>> import pandas as pd
2. >>> dict_series = {'one' : pd.Series([1., 2., 3.], index=['a', 'b', 'c']),
...   'two' : pd.Series([1., 2., 3., 4.], index=['a', 'b', 'c', 'd'])}
3. >>> df = pd.DataFrame(dict_series)
4. >>> df
      one two columns
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0
5. >>> df.shape
(4, 2)
6. >>> df.index
Index(['a', 'b', 'c', 'd'], dtype='object')
7. >>> df.columns
Index(['one', 'two'], dtype='object')
8. >>> list(df.columns)
['one', 'two']
9. >>> dicts_only = {'a':[1,2,3], 'b':[4,5,6]}
```

```

10. >>> dict_df = pd.DataFrame(dict_series)
11. >>> dict_df
      a  b
0  1  4
1  2  5
2  3  6
12. >>> dict_df.index
RangeIndex(start=0, stop=3, step=1)

```

You need to import pandas library ①. Create a dictionary whose values are a series of a one-dimensional arrays ②. You can create a DataFrame from a dictionary of series. Pass *dict_series* dictionary as argument to *DataFrame()* class which returns a DataFrame object ③. Index labels are passed as a list to index attribute. If the number of labels specified in the various series are not the same, then the resulting index will be the union of all the index labels of various series. In DataFrame *df* under column "one", there is no data element associated with index label "d", so NaN will be inserted at that position ④. A number of rows and columns in a DataFrame is obtained using *shape* attribute ⑤. Get the index labels for the DataFrame using *index* attribute ⑥. With *columns* attribute, you get all the columns of the DataFrame ⑦. Get the columns of DataFrame as a list by passing the *columns* attribute as an argument to the *list()* function ⑧. You can create a DataFrame from a dictionary ⑨ without using *index* and *columns* attributes ⑩. For *dict_df* DataFrame ⑪, *a* and *b* are columns and index labels are integers ranging from zero to two ⑫.

Create DataFrame from ndarrays/lists/list of dictionaries

```

1. >>> import numpy as np
2. >>> import pandas as pd
3. >>> dict_ndarrays = {'one': np.random.random(5), 'two': np.random.random(5)}
4. >>> pd.DataFrame(dict_ndarrays)
      one      two
0  0.346580  0.827881
1  0.738850  0.577504
2  0.969715  0.781170
3  0.668432  0.746535
4  0.709333  0.440675
5. >>> pd.DataFrame([1,2,3,4,5], [6,7,8,9,10])
      0  1  2  3  4
rows  0  1  2  3  4  5
      1  6  7  8  9  10

```

6. >>> dict_lists = {'one': [1, 2, 3, 4, 5], 'two': [5, 4, 3, 2, 1]}

7. >>> pd.DataFrame(dict_lists)